

BICO DO DIA

15 — SCRIPT SQL POSTGRESQL

Versão 1.0 — PostgreSQL / Neon — Node.js + React

1. Objetivo

Implementar fisicamente o Modelo Relacional e o Dicionário de Dados do Bico do Dia no PostgreSQL/Neon, com tabelas, chaves, constraints, índices e auditoria.

2. Extensão e schema

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE SCHEMA IF NOT EXISTS bico_do_dia;
SET search_path TO bico_do_dia, public;
```

3. Usuários

```
CREATE TABLE usuarios (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nome VARCHAR(150) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  senha_hash VARCHAR(255) NOT NULL,
  telefone VARCHAR(20),
  tipo_usuario VARCHAR(30) NOT NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'ATIVO',
  email_verificado BOOLEAN NOT NULL DEFAULT FALSE,
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ultimo_login TIMESTAMP
);
```

4. Perfis de cliente e prestador

```
CREATE TABLE clientes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id UUID NOT NULL UNIQUE REFERENCES usuarios(id),
  data_nascimento DATE,
  descricao TEXT,
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE prestadores (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id UUID NOT NULL UNIQUE REFERENCES usuarios(id),
  nome_profissional VARCHAR(150) NOT NULL,
  descricao TEXT,
  experiencia TEXT,
  status VARCHAR(30) NOT NULL DEFAULT 'ATIVO',
  nota_media NUMERIC(3,2) NOT NULL DEFAULT 0 CHECK (nota_media BETWEEN 0 AND 5),
  quantidade_avaliacoes INTEGER NOT NULL DEFAULT 0 CHECK (quantidade_avaliacoes >= 0),
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

5. Endereços

```
CREATE TABLE enderecos (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id UUID NOT NULL REFERENCES usuarios(id),
  cep VARCHAR(10) NOT NULL,
  logradouro VARCHAR(200) NOT NULL,
  numero VARCHAR(20) NOT NULL,
  complemento VARCHAR(100),
  bairro VARCHAR(100) NOT NULL,
  cidade VARCHAR(100) NOT NULL,
  estado VARCHAR(2) NOT NULL,
  latitude NUMERIC(10,7) CHECK (latitude IS NULL OR latitude BETWEEN -90 AND 90),
  longitude NUMERIC(10,7) CHECK (longitude IS NULL OR longitude BETWEEN -180 AND 180),
  principal BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_enderecos_usuario ON enderecos(usuario_id);
CREATE INDEX idx_enderecos_local ON enderecos(cidade, estado, bairro);
```

6. Categorias e especialidades

```
CREATE TABLE categorias (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nome VARCHAR(100) NOT NULL UNIQUE,
  descricao TEXT,
  ativo BOOLEAN NOT NULL DEFAULT TRUE,
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE especialidades (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  categoria_id UUID NOT NULL REFERENCES categorias(id),
  nome VARCHAR(100) NOT NULL,
  descricao TEXT,
  ativo BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (categoria_id, nome)
);
```

7. Especialidades dos prestadores

```
CREATE TABLE prestadores_especialidades (
  prestador_id UUID NOT NULL REFERENCES prestadores(id),
  especialidade_id UUID NOT NULL REFERENCES especialidades(id),
  experiencia_anos INTEGER CHECK (experiencia_anos IS NULL OR experiencia_anos >= 0),
  PRIMARY KEY (prestador_id, especialidade_id)
);
```

8. Serviços

```
CREATE TABLE servicos (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  prestador_id UUID NOT NULL REFERENCES prestadores(id),  
  categoria_id UUID NOT NULL REFERENCES categorias(id),  
  especialidade_id UUID NOT NULL REFERENCES especialidades(id),  
  titulo VARCHAR(150) NOT NULL,  
  descricao TEXT NOT NULL,  
  valor_base NUMERIC(12,2) CHECK (valor_base IS NULL OR valor_base >= 0),  
  unidade_cobranca VARCHAR(30),  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
CREATE INDEX idx_servicos_prestador ON servicos(prestador_id);  
CREATE INDEX idx_servicos_busca ON servicos(categoria_id, especialidade_id, ativo);
```

9. Solicitações de serviço

```
CREATE TABLE solicitacoes_servico (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  cliente_id UUID NOT NULL REFERENCES clientes(id),  
  categoria_id UUID NOT NULL REFERENCES categorias(id),  
  especialidade_id UUID NOT NULL REFERENCES especialidades(id),  
  endereco_id UUID NOT NULL REFERENCES enderecos(id),  
  titulo VARCHAR(150) NOT NULL,  
  descricao TEXT NOT NULL,  
  data_desejada DATE,  
  status VARCHAR(30) NOT NULL DEFAULT 'RASCUNHO',  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CHECK (status IN ('RASCUNHO','ABERTA','EM_NEGOCIACAO','CONTRATADA','CONCLUIDA','CANCELADA'))  
);  
CREATE INDEX idx_solicitacoes_cliente ON solicitacoes_servico(cliente_id,status);  
CREATE INDEX idx_solicitacoes_busca ON solicitacoes_servico(categoria_id,especialidade_id,status);
```

10. Propostas

```
CREATE TABLE propostas (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  solicitacao_id UUID NOT NULL REFERENCES solicitacoes_servico(id),  
  prestador_id UUID NOT NULL REFERENCES prestadores(id),  
  valor NUMERIC(12,2) NOT NULL CHECK (valor >= 0),  
  prazo_execucao INTEGER CHECK (prazo_execucao IS NULL OR prazo_execucao >= 0),  
  descricao TEXT,  
  status VARCHAR(30) NOT NULL DEFAULT 'ENVIADA',  
  validade TIMESTAMP,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CHECK (status IN ('ENVIADA','ACEITA','RECUSADA','EXPIRADA','CANCELADA'))  
);  
CREATE INDEX idx_propostas_solicitacao ON propostas(solicitacao_id,status);  
CREATE INDEX idx_propostas_prestador ON propostas(prestador_id);
```

11. Contratações

```
CREATE TABLE contratacoes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  proposta_id UUID NOT NULL REFERENCES propostas(id),  
  cliente_id UUID NOT NULL REFERENCES clientes(id),  
  prestador_id UUID NOT NULL REFERENCES prestadores(id),  
  solicitacao_id UUID NOT NULL REFERENCES solicitacoes_servico(id),  
  valor_contratado NUMERIC(12,2) NOT NULL CHECK (valor_contratado >= 0),  
  status VARCHAR(30) NOT NULL DEFAULT 'CRIADA',  
  contratado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  concluido_em TIMESTAMP,  
  cancelado_em TIMESTAMP,  
  CHECK (status IN ('CRIADA','AGENDADA','EM_EXECUCAO','CONCLUIDA','CANCELADA'))  
);  
CREATE INDEX idx_contratacoes_cliente ON contratacoes(cliente_id,status);  
CREATE INDEX idx_contratacoes_prestador ON contratacoes(prestador_id,status);
```

12. Agendamentos

```
CREATE TABLE agendamentos (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  contratacao_id UUID NOT NULL REFERENCES contratacoes(id),  
  inicio TIMESTAMP NOT NULL,  
  fim TIMESTAMP NOT NULL,  
  status VARCHAR(30) NOT NULL DEFAULT 'PENDENTE',  
  observacao TEXT,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CHECK (fim > inicio),  
  CHECK (status IN ('PENDENTE','CONFIRMADO','REALIZADO','CANCELADO'))  
);  
CREATE INDEX idx_agendamentos_periodo ON agendamentos(inicio,fim);
```

13. Avaliações

```
CREATE TABLE avaliacoes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  contratacao_id UUID NOT NULL REFERENCES contratacoes(id),  
  cliente_id UUID NOT NULL REFERENCES clientes(id),  
  prestador_id UUID NOT NULL REFERENCES prestadores(id),  
  nota SMALLINT NOT NULL CHECK (nota BETWEEN 1 AND 5),  
  comentario TEXT,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE (contratacao_id, cliente_id)  
);  
CREATE INDEX idx_avaliacoes_prestador ON avaliacoes(prestador_id);
```

14. Favoritos e notificações

```
CREATE TABLE favoritos (  
  cliente_id UUID NOT NULL REFERENCES clientes(id),  
  prestador_id UUID NOT NULL REFERENCES prestadores(id),  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (cliente_id, prestador_id)  
);  
  
CREATE TABLE notificacoes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  usuario_id UUID NOT NULL REFERENCES usuarios(id),  
  titulo VARCHAR(150) NOT NULL,  
  mensagem TEXT NOT NULL,  
  tipo VARCHAR(50) NOT NULL,  
  lida BOOLEAN NOT NULL DEFAULT FALSE,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  lida_em TIMESTAMP  
);  
CREATE INDEX idx_notificacoes_usuario ON notificacoes(usuario_id,lida);
```

15. Denúncias, moderação e auditoria

```
CREATE TABLE denuncias (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  denunciante_id UUID NOT NULL REFERENCES usuarios(id),  
  usuario_denunciado_id UUID NOT NULL REFERENCES usuarios(id),  
  motivo VARCHAR(100) NOT NULL,  
  descricao TEXT NOT NULL,  
  status VARCHAR(30) NOT NULL DEFAULT 'ABERTA',  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  atualizado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CHECK (status IN ('ABERTA','EM_ANALISE','RESOLVIDA','ARQUIVADA'))  
);  
  
CREATE TABLE moderacoes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  denuncia_id UUID NOT NULL REFERENCES denuncias(id),  
  administrador_id UUID NOT NULL REFERENCES usuarios(id),  
  decisao VARCHAR(100) NOT NULL,  
  observacao TEXT,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE auditorias (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  usuario_id UUID REFERENCES usuarios(id),  
  acao VARCHAR(100) NOT NULL,  
  entidade VARCHAR(100) NOT NULL,  
  entidade_id UUID,  
  dados_anteriores JSONB,  
  dados_novos JSONB,  
  ip INET,  
  user_agent TEXT,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
CREATE INDEX idx_denuncias_status ON denuncias(status);  
CREATE INDEX idx_auditorias_usuario_data ON auditorias(usuario_id,criado_em);
```

16. Trigger de atualização

```
CREATE OR REPLACE FUNCTION atualizar_timestamp()  
RETURNS TRIGGER AS $$  
BEGIN  
  NEW.atualizado_em = CURRENT_TIMESTAMP;  
  RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE OR REPLACE TRIGGER trg_usuarios_atualizado  
BEFORE UPDATE ON usuarios FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_clientes_atualizado  
BEFORE UPDATE ON clientes FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_prestadores_atualizado  
BEFORE UPDATE ON prestadores FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_servicos_atualizado  
BEFORE UPDATE ON servicos FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_solicitacoes_atualizado  
BEFORE UPDATE ON solicitacoes_servico FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_propostas_atualizado  
BEFORE UPDATE ON propostas FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_contratacoes_atualizado  
BEFORE UPDATE ON contratacoes FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_agendamentos_atualizado  
BEFORE UPDATE ON agendamentos FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();  
  
CREATE OR REPLACE TRIGGER trg_denuncias_atualizado  
BEFORE UPDATE ON denuncias FOR EACH ROW EXECUTE FUNCTION atualizar_timestamp();
```

17. Dados iniciais

```
INSERT INTO categorias (nome, descricao) VALUES  
( 'Limpeza', 'Serviços de limpeza residencial e comercial'),  
( 'Manutenção', 'Serviços gerais de manutenção'),  
( 'Elétrica', 'Serviços elétricos'),  
( 'Hidráulica', 'Serviços hidráulicos'),  
( 'Informática', 'Serviços de informática e tecnologia')
```

18. Controle de concorrência e agendamento

O banco registra intervalos de início e fim. A API Node.js deverá validar conflitos dentro de transações. Quando a regra de disponibilidade estiver fechada, recomenda-se reforçar a proteção com EXCLUDE USING GIST e range de tempo, garantindo que um prestador não possua horários sobrepostos.

19. Implantação no Neon

Criar o projeto PostgreSQL no Neon; configurar a connection string em variável de ambiente; executar o script em desenvolvimento; validar constraints e índices; transformar alterações posteriores em migrations versionadas; executar testes de integração antes da produção.

20. Segurança

Nunca armazenar senha em texto puro. O campo senha_hash deve receber somente hash seguro. JWT/OAuth pertence à camada de autenticação. Connection strings e secrets não devem entrar no Git. Auditorias não devem armazenar tokens ou senhas.

21. Rastreabilidade

Documento 12 — MER → Documento 13 — Modelo Relacional → Documento 14 — Dicionário de Dados → Documento 15 — Script SQL. O script é a primeira versão física do banco e deverá evoluir por migrations.

22. Próximo artefato

Documento 16 — Arquitetura do Sistema, detalhando frontend React, backend Node.js, API REST, autenticação, banco Neon, integrações e estratégia de escalabilidade.